✅ SYSTEM MANUAL: ONLINE EXAMINATION SYSTEM

---

1. 🎯 OBJECTIVE / VISION

The purpose of this project is to develop a secure and user-friendly **Online Examination System**. The system allows students to register, verify their identity through facial recognition, and take exams in a controlled environment. Key security features like **real-time image verification** and **automated proctoring** are built in to prevent cheating and ensure exam integrity.

The system streamlines the process for **examiners/instructors** to create and manage exams while providing **students** with instant feedback and results. The ultimate vision is to make online exams more **accessible, secure, and reliable** for all users.

---

2. 👥 USERS OF THE SYSTEM

🎓 Students

- Register for exams with personal and facial identity verification.
- Access and complete exams within the allocated schedule.
- Receive results and performance feedback after evaluation.

👨‍🏫 Examiners / Instructors

- Create, schedule, and manage exams and questions.
- Monitor student activity and verify proctoring logs/images.
- Evaluate responses and finalize scores.
- Generate student performance reports.

👨‍💻 System Administrators

- Manage user accounts and enforce system security.
- Monitor database and server performance.
- Add, remove, or manage lecturer accounts.

👮 Proctors

- Monitor students live during exams.
- Verify student identity and flag suspicious activity.

🛠 Technical Support

- Provide technical support to students and staff.
- Troubleshoot system issues and ensure uptime.

---

3. ⚙ SYSTEM FEATURES

📝 Student Registration

- Sign up using name, email, password, and profile image (used for face verification).

🔐 Login & Authentication

- Secure login with password.
- Facial verification required for exam entry.

📋 Exam Management

- List of available exams for each enrolled student.
- Exams have due dates, time limits, and maximum attempts.
- Supports **multiple-choice**, **short-answer**, and **subjective** question types.

🛡 Proctoring & Security

- Real-time facial verification during the exam.
- Detection of:
  - No face
  - Multiple faces
  - Background mismatch
- If identity mismatch or cheating is detected, the student is logged out and attempt is flagged.

📊 Results & Feedback

- **Auto-grading** for objective questions.
- **Manual marking** for subjective answers.
- Results shown upon completion (immediate or post-review).

---

4. 🧠 TECHNOLOGY STACK

Frontend:

- HTML5, CSS3 (or optionally ReactJS & Tailwind CSS if used)

Backend:

- **Python 3.10+**
- **Flask** (Web Framework)
- **Flask-Login** (Authentication)
- **Flask-Mail** (Email Verification/Reset)

Database:

- **PostgreSQL** (or SQLite for local development)
- **SQLAlchemy ORM**

Face Detection:

- Face recognition libraries (e.g., `face_recognition` or OpenCV)
- Face encodings stored for verification

File & Image Storage:

- Local file system or **Cloudinary** for profile pictures

Email & Communication:

- SMTP (Gmail or Brevo) for sending email verifications and reset codes

---

5. 💻 SYSTEM REQUIREMENTS

- Python 3.10+
- PostgreSQL (or SQLite for local dev)
- Flask & Flask extensions
- Web browser (Chrome, Firefox, or Edge)
- Internet access (for email verification and online deployment)

---

6. 📅 IMPLEMENTATION PLAN

Phase 1: Planning & Design

- Define database schema
- Design user interface (UI) wireframes
- Finalize exam flow and security features

Phase 2: Development

- Implement backend (Flask, SQLAlchemy)
- Develop frontend (HTML/CSS or React/Tailwind)
- Integrate email, proctoring, and face verification

Phase 3: Testing & Deployment

- Unit and integration testing
- Bug fixing and optimization
- Deploy system on cloud (e.g., Heroku, Render, or VPS)

---

7. 💡 EXPECTED BENEFITS

- Improves exam integrity with **face monitoring** and **background checks**.
- Enhances teaching productivity by automating exam evaluation.
- Provides real-time feedback and analytics.
- Reduces exam fraud and impersonation cases.

---

8. 🛠 INSTALLATION INSTRUCTIONS

🔧 Step-by-Step Setup

1. **Clone the Repository:**
2. Create Virtual Environment:
3. Install Python Dependencies:
4. Configure Environment Variables:
5. Initialize Database:
6. Run the Application:
7. Access the System: Navigate to: `http://localhost:5000/`